HƯỚNG DẪN THỰC HÀNH · MÃ ĐÃ HOÀN CHỈNH

Từ một bảng số đến một lời giải kiểm chứng được

Nguyễn Bích Vân – IAI–UET–VNU

Học phần: Biểu diễn tri thức và tìm kiếm nâng cao

Không có hàm bỏ trống hoặc `NotImplementedError`. Chạy ví dụ trước, sau đó thay dữ liệu. Python 3.10 trở lên; chỉ dùng thư viện chuẩn. Không cần NumPy, NetworkX, cài GPU hoặc `pip install`.

1. Chạy được trong VS Code trên Windows

Giải nén **toàn bộ** gói. File > Open Folder: mở thư mục `Nhom1_8Puzzle`, trong đó có trực tiếp `run.py`. Chọn Python/Anaconda đã cài, mở Terminal > New Terminal.

```
python kiem_tra_moi_truong.py
python chay_mau.py
python astar_toi_gian.py
python -m unittest -v test_mau
```

Mỗi lệnh chạy riêng. **Không gõ `cd python`** vì không có thư mục con như vậy. Nếu đang ở thư mục khác, chuyển tới thư mục đã giải nén hoặc chạy bằng đường dẫn đầy đủ trong dấu ngoặc kép, ví dụ

```
python "D:\Bieu dien tri thuc\Nhom1_8Puzzle\run.py" --case DEMO
```

Nếu máy dùng launcher, có thể thay python bằng `py -3`. Khi dấu nhắc là `>>>`, gõ `exit()` để trở về Terminal trước. `can't open file` là sai đường dẫn/tên tệp, chưa phải thuật toán sai. `No module named puzzle` thường do thiếu helper hoặc di chuyển riêng một tệp.

Ví dụ này đã giải để học, không phải đáp án bài nộp

Với D trong `chay_mau.py`, một đường là D,R: 0 đi xuống rồi sang phải. Unit: 2 bước, cost 2. Directional: vẫn 2 bước, cost $1 + 2 = 3$. Hãy nhìn ba bảng in ra và kiểm tra đúng vị trí số 0; sau đó mở `output/demo.json`.

2. Thay dữ liệu mà không sửa thuật toán

Cách A: sửa một khối trong chay_mau.py

```
START = (1, 2, 3, 4, 0, 6, 7, 5, 8)
MODEL = "unit"
HEURISTIC = "manhattan"
# GOAL duoc import o tren; co the gan lai mot tuple khac.
result = solve(START, GOAL, algorithm="astar",
               heuristic=HEURISTIC, model=MODEL, trace_limit=6)
```

Thay START là đổi bàn. Thay GOAL là đổi đích. Thay MODEL để đổi chi phí. Không đổi riêng con số trong báo cáo; phải chạy lại rồi ghi nhận kết quả mới.

Cách B: dữ liệu JSON cho hai bàn của nhóm

```
{
  "goal": [1,2,3,4,5,6,0,7,8],
  "cases": {
    "N1": [1,2,3,4,5,6,7,0,8],
    "N2": [1,2,3,4,0,6,7,5,8]
  }
}
```

Đây chỉ là ví dụ cú pháp dễ; **nhóm phải thay hai bàn mới đủ độ khó theo đề**. Tên trường giữ nguyên. Dùng ngoặc kép, không dùng tuple Python trong JSON; không có dấu phẩy thừa.

```
python run.py --data du_lieu_nhom.json --case N1 --algo astar --heuristic weighted --model directional --tag
    thu_N1
python run.py --data du_lieu_nhom.json --suite --tag tu_tao
```

Đường `-data` tương đối tính từ Terminal; khi không ở đúng thư mục thì dùng đường dẫn đầy đủ cho cả tệp JSON.

Cách C: sinh dữ liệu bằng bước hợp lệ từ goal

```
python tao_du_lieu.py --seed 2026 --steps 16 --goal "1,2,3,4,5,6,0,7,8" --out du_lieu_nhom.json
```

Mỗi N có một lịch sử từ goal. Vì mọi phép đổi chỗ có thể đảo, dữ liệu sinh khả giải. Tuy nhiên, 16 bước xáo trộn *không* chứng minh cần ít nhất 16 bước để giải: có thể có chu trình/đường tắt. Dùng BFS kiểm số bước tối ưu 6–24, xác nhận N1 khác N2 và khác dữ liệu chung; nếu chưa đạt, đổi seed.

3. Đọc biểu diễn trạng thái và heuristic

Tuple chín số: vì sao cần bất biến?

Tuple dùng làm khóa `best_g`. Mỗi lần chuyển phải tạo list tạm để hoán đổi, sau đó tạo tuple mới. Không sửa bàn cha hoặc một list được nhiều nhánh cùng giữ.

```
zero = state.index(0)
r, c = divmod(zero, 3)
# Vi du thu hanh dong U:
if r > 0:
    new = list(state)
    target = zero - 3
    new[zero], new[target] = new[target], new[zero]
    child = tuple(new)
```

Tính h dựa trên đích được truyền vào

```
position = {tile: divmod(i, 3) for i, tile in enumerate(goal)}
result = 0
for i, tile in enumerate(state):
    if tile == 0:
        continue
    r, c = divmod(i, 3)
    rg, cg = position[tile]
    result += abs(r-rg) + abs(c-cg)
```

Đổi hệ số trước $|c - c_g|$ thành 2 chỉ trong h3/directional. Vị trí của một quân tính bằng chỉ số quân đó trong goal, không bằng công thức mặc định nếu goal đã đổi.

Tên hàm trong mã

Tên	Vai trò
`validate_state`	Kiểm đúng chín số nguyên 0–8, không trùng/thiếu; không chấp nhận bool.
`is_solvable`	Đếm parity theo goal; chỉ dành cho bàn 3x3.
`neighbors`	Sinh hành động, tuple con, giá cạnh theo model/order.
`make_heuristic`	Trả hàm h có sẵn vị trí goal; bỏ 0.
`check_path`	Kiểm đầu–cuối, mỗi bước và tổng giá, không chỉ độ dài.

Dữ liệu sai gây `ValueError`; dữ liệu hợp lệ nhưng khác parity là `unsolvable`. Hai trường hợp này không giống nhau. Một giới hạn tài nguyên bị chạm lại là trường hợp thứ ba.

4. A* tối giản – mã hoàn chỉnh để đọc

Tệp `astar_toi_gian.py` đã chạy được cùng `puzzle.py`. Dưới đây là lõi, không phải phần còn khuyết cần tự điền:

```python
from heapq import heappush, heappop
from itertools import count
from puzzle import GOAL, neighbors, make_heuristic, validate_state, is_solvable


def astar(start, goal=GOAL, model="unit", heuristic="manhattan"):
    start, goal = validate_state(start), validate_state(goal)
    if not is_solvable(start, goal):
        return None
    h = make_heuristic(goal, heuristic, model)
    serial = count()
    frontier = [(h(start), next(serial), 0, start)]
    best_g, parent = {start: 0}, {start: None}
    while frontier:
        f, _, g, state = heappop(frontier)           # 1. Chon f nho nhat
        if g != best_g[state]:                       # 2. Bo ban ghi cu
            continue
        if state == goal:                            # 3. Chi dung o ay
            path = []
            while state is not None:
                path.append(state)
                state = parent[state]
            return list(reversed(path)), g
        for action, child, cost in neighbors(state, model):
            new_g = g + cost                         # 4. Gia toi con
            if new_g < best_g.get(child, float("inf")):
                best_g[child], parent[child] = new_g, state
                heappush(frontier, (new_g+h(child), next(serial), new_g, child))
    return None
```

Đọc thành lời: lấy nút có tổng "đã trả + còn ước lượng" nhỏ nhất; bỏ bản ghi cũ; chỉ dừng khi chính nút đó là đích; hỏi các con; nếu tìm được giá tốt hơn thì cập nhật. Biến `serial` đảm bảo FIFO ở f bằng nhau. Trường g trong heap là snapshot, không phải tra lại một giá mới rồi gán nhầm cho đường cũ.

5. Từ mã ngắn sang bản có thống kê

Một node gắn với một đường

Bản `search.py` dùng `Node(state,g,depth,parent,action)` bất biến. Hai node có thể trỏ cùng state nhưng là hai đường khác nhau. Heap chỉ giữ mã node; parent là mã node cha, nên dựng lại đường không bị ảnh hưởng bởi việc một state sau đó có đường tốt hơn.

best_g không đồng nghĩa đã xử lý xong

`best_g[s]` là giá thấp nhất *đã phát hiện* tới s. Nó có thể giảm trước khi s được lấy ra. Bản ghi cũ vẫn trong heap; nếu g của bản ghi khác best_g hiện tại thì bỏ khi pop. `expanded_g` lưu giá ở lần đã mở rộng, giúp hỗ trợ mở lại nếu xuất hiện đường rẻ hơn. Các heuristic chuẩn của đề là nhất quán, nên bản chuẩn không cần mở lại trạng thái đã mở rộng.

Khác biệt của BFS

BFS dùng deque và đánh dấu khi đưa vào; không chọn theo cost. Cost vẫn được cộng để *báo cáo* chi phí thật của đường BFS. Đổi cost không tự biến BFS thành UCS.

Số đếm đặt ở đâu?

Sự kiện	Thống kê
Đưa gốc vào biên	pushed=1, max_frontier=1.
Mỗi lần lấy ra vật lý	popped tăng 1; nếu lỗi thời thì stale_pops tăng và bỏ.
Gặp đích ở lần lấy hợp lệ	Trả lời giải; không tăng expanded, không sinh con.
Sinh con cho nút chưa phải đích	expanded tăng 1; mỗi con hợp lệ tăng generated.
Con mới/cải thiện được đưa vào	pushed tăng 1; cập nhật max_frontier sau vòng sinh con.
Chạm max_expanded	status=limit, chưa có lời giải hoàn chỉnh; không ghi vô nghiệm.

Đọc max_frontier cho đúng

Đây là số bản ghi biên lớn nhất, không phải toàn bộ bộ nhớ. Node đã tạo, best_g, expanded_g và trace cũng dùng bộ nhớ. Mảng heap không được sắp hoàn toàn: chỉ phần tử đầu đảm bảo nhỏ nhất; muốn xem thứ tự pop đầy đủ cần sắp bản sao, như bảng trace đã làm.

6. Chạy từng bước và tạo ma trận kết quả

Dừng trực tiếp trong thuật toán

```
python run.py --case DEMO --algo astar --heuristic manhattan --step --tag demo_step
python run.py --case HAND --algo astar --heuristic manhattan --trace 6 --tag tinh_tay
```

`-step` dừng *trước khi sinh con* của mỗi nút hợp lệ; nhấn Enter trong Terminal để tiếp. Không phải phát lại lời giải đã tính xong. `-trace 6` chỉ lưu sáu bản ghi đầu; thuật toán vẫn chạy tới đích. Dùng Ctrl+C để dừng, khi ấy chưa có kết quả hoàn chỉnh.

Muốn theo dõi từng dòng: mở `astar_toi_gian.py`; đặt breakpoint ở heappop hoặc cập nhật best_g; chọn Python Debugger: Debug Python File. F10 đi qua dòng; F11 đi vào hàm; Variables hiện biến, Call Stack hiện lời gọi. Khi dùng debugger, không cần bật step cùng lúc.

Chạy đủ dữ liệu chung và riêng

```
python run.py --suite --tag bat_buoc
python run.py --data du_lieu_nhom.json --suite --tag tu_tao
python run.py --case HAND --order RDLU --trace 6 --tag doi_thu_tu
```

Lệnh đầu tạo 36 dòng, lệnh thứ hai tạo 18 dòng khi JSON có đúng hai case. Suite không cho trace/step để tránh tạo log quá lớn. Mỗi kết quả JSON có path/actions; CSV chỉ giữ số liệu. Tên tag khác nhau tránh ghi đè các lần cần giữ. Output luôn cạnh run.py.

Không bắt buộc đo thời gian

```
python run.py --suite --repeat 3 --tag timing
```

Đo trong cùng máy/Python/cấu hình, không có trace/step. elapsed_ms lấy trung vị; có thể khác đáng kể giữa các máy. Không so lần có in nhiều log với lần không có log.

Đọc status trước số liệu

`solved`: có đường hoàn chỉnh; `unsolvable`: bị loại theo parity; `limit`: chưa xong vì ngân sách; `exhausted`: hết biên. Không thay None thành 0 rồi đưa vào bảng như chi phí tối ưu.

7. Viết test và điền báo cáo

Chạy test mẫu để kiểm cài đặt, rồi sao chép `test_nhom_mau.py` thành `test_nhom.py` và viết ít nhất 10 test có dữ liệu/khẳng định riêng. Ví dụ mới phải có kỳ vọng tự giải thích được:

```
class MyTests(unittest.TestCase):
    def test_move_vertical(self):
        start = (1,2,3,4,5,0,7,8,6)
        r = solve(start, model="directional")
        self.assertEqual(r["cost"], 1)
        self.assertEqual(r["actions"], ["D"])
```

Ví dụ trên chỉ minh họa cú pháp, không tự tính là test mới nếu sao nguyên. `assertEqual` so với dự đoán; lệnh print đơn thuần không kiểm một thuộc tính đúng/sai.

```
python -m unittest -v test_nhom
python -m unittest -v test_nhom > output/ket_qua_test.txt 2>&1
```

Trong Windows PowerShell có thể dùng `*> output/ket_qua_test.txt` để gom các luồng. Tạo output trước bằng chạy demo/suite. Kiểm log có số test và OK hoặc lỗi thật; không sửa log để trông đẹp.

Điền mẫu báo cáo

Chọn Word hoặc LaTeX, không cần điền cả hai. Mỗi nhận xét theo cấu trúc: **cấu hình nào – quan sát gì – bằng chứng tệp/dòng – giải thích – giới hạn**. Giữ dự đoán tính tay trước khi nhìn code. Bảng chuẩn không điền sẵn đáp số trong mẫu sinh viên.

Sửa thuật toán khi phát hiện lỗi

Giữ bản lỗi riêng, ghi đầu vào nhỏ gây lỗi và test trước/sau sửa. Không âm thầm đổi data cho khớp kết quả mong muốn. Khi dùng mã mẫu không sửa lõi, vẫn cần ghi rõ phần dùng lại và minh chứng hiểu qua test/giải thích.

8. Đóng gói và tập bảo vệ

Đổi `README_NHOM_MAU.md` thành `README_NHOM.md`, điền nguồn/dữ liệu/lệnh; đổi `DongGop_Mau.csv` thành `DongGop.csv`, điền đúng đóng góp. Đặt báo cáo `BaoCao.pdf`, tính tay `TinhTay.pdf` và slide `ThuyetTrinh.pptx` (hoặc PDF) cạnh các tệp Python. Sau khi đã chạy và lưu log:

```
python dong_goi.py --group Nhom01
```

Script kiểm tên tệp, 36/18 dòng và trạng thái solved của các lần chạy yêu cầu. **Nó không kiểm xem báo cáo giải thích đúng, test có độc lập hay mọi thành viên hiểu mã.** Giải nén ZIP mới ở thư mục khác, chạy lại README rồi mới nộp lên website học phần.

Bảo vệ 18 phút: không đọc code từ đầu đến cuối

5 phút nói mô hình/A*/heuristic/kết quả; 3 phút chạy một trường hợp nhóm tự tạo; 10 phút để từng người dự đoán một thay đổi hoặc giải thích một test. Bằng chứng cần chuẩn bị: một board; một bảng cost và expanded; một trace nhỏ; một test bắt lỗi. Tập đưa chuột vào dòng chọn nút, best_g, kiểm goal, chi phí cạnh và hàm h.

Nguồn tra cú pháp và thao tác

Các nguồn dưới chỉ để tra công cụ, không thay cho lời giải toán:

1. Python, *heapq – Priority queue implementation notes*:
   `https://docs.python.org/3/library/heapq.html`
2. Python, *unittest – Unit testing framework*:
   `https://docs.python.org/3/library/unittest.html`
3. Microsoft, *Python debugging in VS Code*:
   `https://code.visualstudio.com/docs/python/debugging`